End-to-End Machine Learning Project

Part 7: prepare the data – transformation pipelines and custom transformers

Marcus Birkenkrahe

COSC 3430 · Artificial Intelligence · Augustana University

Based on Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Ch. 2

Fall 2026

Roadmap

Ten steps, over several code-along lectures:

1. Look at the big picture
2. Get the data
3. Explore and visualize the data
4. Create the test data
5. Prepare the data for learning
6. Select a model and train it
7. Fine-tune the model
8. Present your solution
9. Launch the production system
10. Monitor and maintain it

Today: step 5, last part – transformation pipelines and custom transformers.
Next time: select a model and train it.

Where part 6 left off

1. Each preparation step is a transformer: `SimpleImputer`, `OneHotEncoder`, `MinMaxScaler`, `StandardScaler`
2. Heavy tails call for a log transform, multimodal features for RBF similarity (`rbf_kernel`)
3. So far, we called each transformer by hand, one after the other, and for the numerical and categorical columns separately
4. Today: chain the steps into **pipelines**, apply them to the right **columns**, and write our own **custom transformer**

A small example

→ **Switch to editor · code-along 1**

Two numeric features, each missing one value:

```
toy = pd.DataFrame({
    "age":    [25, 32, np.nan, 40, 28],
    "income": [50000, 64000, 58000,
               np.nan, 72000],
})
print(toy)
print(toy.describe())
```

What do you see?

→ Massive scale differences

→ Missing values

```
    age   income
0  25.0  50000.0
1  32.0  64000.0
2   NaN  58000.0
3  40.0      NaN
4  28.0  72000.0
         age        income
count   4.00      4.000000
mean   31.25  61000.000000
std     6.50   9309.493363
min    25.00  50000.000000
max    40.00  72000.000000
```

Pipeline: impute, then scale

→ **Switch to editor · code-along 2**

Each step is a (name, transformer) pair, applied in order:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])

toy_prepared = num_pipeline.fit_transform(toy)
print(toy_prepared.round(2))
```

```
[[-1.19 -1.53]
 [ 0.2   0.42]
 [-0.2  -0.42]
 [ 1.78  0.  ]
 [-0.59  1.53]]
```

`fit_transform()` does both steps: the imputer fills the gaps with each column's median, then the scaler standardizes the complete result.

The same steps by hand

→ **Switch to editor · code-along 3**

```
imputer = SimpleImputer(strategy="median")
toy_imputed = imputer.fit_transform(toy)
print(toy_imputed)

scaler = StandardScaler()
toy_scaled = scaler.fit_transform(toy_imputed)
print(np.round(toy_scaled, 2))
```

The `Pipeline` calls `fit_transform()` on each step in turn and passes the output forward – it saves you from writing this out by hand.

```
[[2.5e+01 5.0e+04]
 [3.2e+01 6.4e+04]
 [3.0e+01 5.8e+04]
 [4.0e+01 6.1e+04]
 [2.8e+01 7.2e+04]]
[[-1.19 -1.53]
 [ 0.2   0.42]
 [-0.2  -0.42]
 [ 1.78  0.  ]
 [-0.59  1.53]]
```

The rules of a `Pipeline`

- The constructor takes a list of name/estimator pairs (2-tuples): the sequence of steps
- All estimators must be **transformers** (they have `fit_transform()`) – except the last one, which can be anything
- A natural last step is a **predictor**: an estimator with `fit()` and `predict()` instead of `transform()`
- Example: use `toy.age` as the feature and `income` as the target, with linear regression as the last step

A predictor as the last step

→ **Switch to editor · code-along 4**

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error

train = toy.dropna(subset=["income"])
X = train[["age"]]
y = train["income"]

full_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("predict", LinearRegression())])

full_pipeline.fit(X, y)
predictions = full_pipeline.predict(X)
print(np.round(predictions, 0))
rmse = root_mean_squared_error(y, predictions)
print(f"RMSE: {rmse:,.0f}")
```

```
[55091. 67818. 60545. 60545.]
RMSE: 6,674
```

`X` must be 2D: `train[["age"]]`, not `train["age"]`.

Lousy prediction – relative error > 10%. Why?

→ We evaluated on the same 4 rows the model was trained on (no train/test split), and the sample was too small to begin with.

make_pipeline, and looking inside

→ **Switch to editor · code-along 5**

Don't want to name the steps? `make_pipeline` uses the class names in lower case:

```
from sklearn.pipeline import make_pipeline
pipeline = make_pipeline(SimpleImputer(strategy="median"),
                         StandardScaler(),
                         LinearRegression())
print(pipeline)
print(pipeline.steps)
print(len(pipeline))
print(pipeline[0])
print(pipeline[-1])
```

```
Pipeline(steps=[('simpleimputer', SimpleImputer(strategy='median')),
                ('standardscaler', StandardScaler()),
                ('linearregression', LinearRegression())])
```

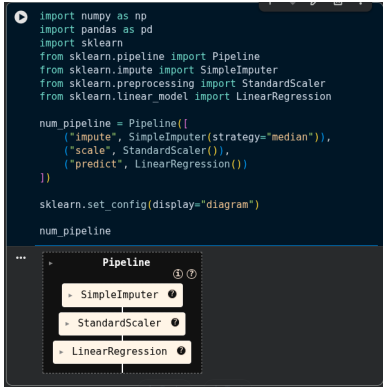
Pipeline diagram in a notebook

In Colab (or any Jupyter notebook):

```
import sklearn
sklearn.set_config(display="diagram")

num_pipeline
```

- One box per step – click it to see that step's parameters
- num_pipeline must be the **last line of the cell**, not wrapped in print(): that triggers the notebook's rich display, as for a DataFrame
- Notebooks only – a plain Python session prints text



A pipeline for the housing data

→ **Switch to editor · code-along 6**

```
num_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler())
housing_num_prepared = \
    num_pipeline.fit_transform(housing_num)
print(housing_num_prepared[:2].round(2))
```

```
[[-1.42  1.01  1.86  0.31
   1.37  0.14  1.39 -0.94]
 [ 0.6  -0.7   0.91 -0.31
  -0.44 -0.69 -0.37  1.17]]
```

The output is a NumPy array. Restore the names and index:

```
df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared,
    columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)
print(df_housing_num_prepared
      .describe().round(2))
```

	longitude	...	median_income
mean	-0.00	...	0.00
std	1.00	...	1.00
min	-2.39	...	-1.78
max	2.54	...	5.88

ColumnTransformer: numerical and categorical columns

→ **Switch to editor · code-along 7**

So far, we handled numerical and categorical columns separately. ColumnTransformer applies each pipeline to its own columns:

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude",
    "housing_median_age", "total_rooms",
    "total_bedrooms", "population",
    "households", "median_income"]
cat_attribs = ["ocean_proximity"]

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))
```

A list of 3-tuples: name, transformer, target columns:

```
preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs)
])
```

- Categorical columns: impute the **most frequent** category, then one-hot encode
- handle_unknown="ignore": unknown categories become all zeros

Let scikit-learn pick the columns

→ **Switch to editor · code-along 8**

```
from sklearn.compose import (
    make_column_selector,
    make_column_transformer)

preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(
        dtype_include=np.number)),
    (cat_pipeline, make_column_selector(
        dtype_include=object)),
)
housing_prepared = \
    preprocessing.fit_transform(housing)
print(housing_prepared.shape)
print(preprocessing.get_feature_names_out())
```

```
(16512, 13)
['pipeline-1__longitude'
 'pipeline-1__latitude'
 ...
 'pipeline-1__median_income'
 'pipeline-2__ocean_proximity_<1H OCEAN'
 ...
 'pipeline-2__ocean_proximity_NEAR OCEAN']
```

- No names needed: the steps are called `pipeline-1`, `pipeline-2`
- 8 scaled numerical + 5 one-hot columns = 13 features

A custom transformer: `ClusterSimilarity`

```python
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.metrics.pairwise import rbf_kernel
from sklearn.cluster import KMeans
class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state
    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters,
                              random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self
    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_,
                          gamma=self.gamma)
    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity"
                for i in range(self.n_clusters)]
```

→ **Switch to editor · code-along 9**

- `fit()`: KMeans finds the main clusters; always return `self`
- `transform()`: RBF similarity of each district to each cluster center – one feature per cluster
- Higher = closer to that centroid
- `TransformerMixin` adds `fit_transform()`, `BaseEstimator` adds `get_params()`, `set_params()`

Use it like any other transformer

→ **Switch to editor · code-along 10**

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]])
print(similarities.shape)
print(similarities[:3].round(2))
```

```
(16512, 10)
[[0.46 0.   0.08 0.   0.   0.   0.   0.98 0.   0.  ]
 [0.   0.96 0.   0.03 0.04 0.   0.   0.   0.11 0.35]
 [0.34 0.   0.45 0.   0.   0.   0.01 0.73 0.   0.  ]]
```

The first district is close to cluster 7 (0.98), the second to cluster 1 (0.96): proximity to a cluster says more about price than raw coordinates.

The final transformer: what it does

- Impute missing numerical values with the **median**
- Impute missing categorical values with the **most frequent** category
- **One-hot encode** the categorical feature
- Add the **ratio** features `bedrooms_ratio`, `rooms_per_house`, `people_per_house` – they correlate with `median_house_value`
- Add **cluster similarity** features – more useful than latitude and longitude
- Replace heavy-tailed features by their **logarithm**
- **Standardize** all numerical features
- Exclude `housing_median_age` from all of these transformations

Reusable pieces: FunctionTransformer

→ **Switch to editor · code-along 11**

```
from sklearn.preprocessing import FunctionTransformer

def column_ratio(X):
    return X[:, [0]] / X[:, [1]]
def ratio_name(function_transformer,
               feature_names_in):
    return ["ratio"]
def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio,
            feature_names_out=ratio_name),
        StandardScaler())
```

```
log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log,
        feature_names_out="one-to-one"),
    StandardScaler())

cluster_simil = ClusterSimilarity(
    n_clusters=10, gamma=1.,
    random_state=42)

default_num_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler())
```

FunctionTransformer turns a plain function into a transformer (nothing to learn in fit()).

The final ColumnTransformer

→ **Switch to editor · code-along 12**

```
preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                           "households", "median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
], remainder=default_num_pipeline)
```

- Each ratio pipeline takes two columns and returns one: their ratio
- `remainder`: what happens to all columns not listed – here, only `housing_median_age`

Why is housing_median_age left out?

Why does `housing_median_age` get only median imputation and standard scaling?

- → The other features have specific problems: raw counts carry little signal on their own – the **ratio** does; `population` etc. are heavily skewed – **log**; raw coordinates are not linearly related to price – proximity to **clusters** is
- → `housing_median_age` has none of these issues: not a raw count, not heavily skewed, not geographic. It is already well-behaved, so impute + scale via `remainder` is enough.

24 features, ready for training

→ **Switch to editor · code-along 13**

```
housing_prepared = preprocessing.fit_transform(housing)
print(housing_prepared.shape)
print(preprocessing.get_feature_names_out())
```

```
(16512, 24)
['bedrooms__ratio' 'rooms_per_house__ratio' 'people_per_house__ratio'
 'log__total_bedrooms' 'log__total_rooms' 'log__population'
 'log__households' 'log__median_income' 'geo__Cluster 0 similarity'
 ... 'geo__Cluster 9 similarity' 'cat__ocean_proximity_<1H OCEAN'
 'cat__ocean_proximity_INLAND' 'cat__ocean_proximity_ISLAND'
 'cat__ocean_proximity_NEAR BAY' 'cat__ocean_proximity_NEAR OCEAN'
 'remainder__housing_median_age']
```

3 ratios + 5 logs + 10 cluster similarities + 5 one-hot columns + 1 remainder = 24. Each name starts with the name of its step.

What we built

- **`Pipeline`**: a sequence of (name, estimator) steps – transformers, with any estimator (e.g. a predictor) last; `make_pipeline` names the steps for you
- **`ColumnTransformer`**: one pipeline per group of columns; or `make_column_transformer` with `make_column_selector`
- **Custom transformers**: `FunctionTransformer` for a plain function; a class with `fit()` (returns `self`) and `transform()` when something has to be learned, like `ClusterSimilarity`
- **The final transformer**: ratios, logs, cluster similarities, one-hot encoding and scaling in one object – 24 features

Next: select a model and train it.